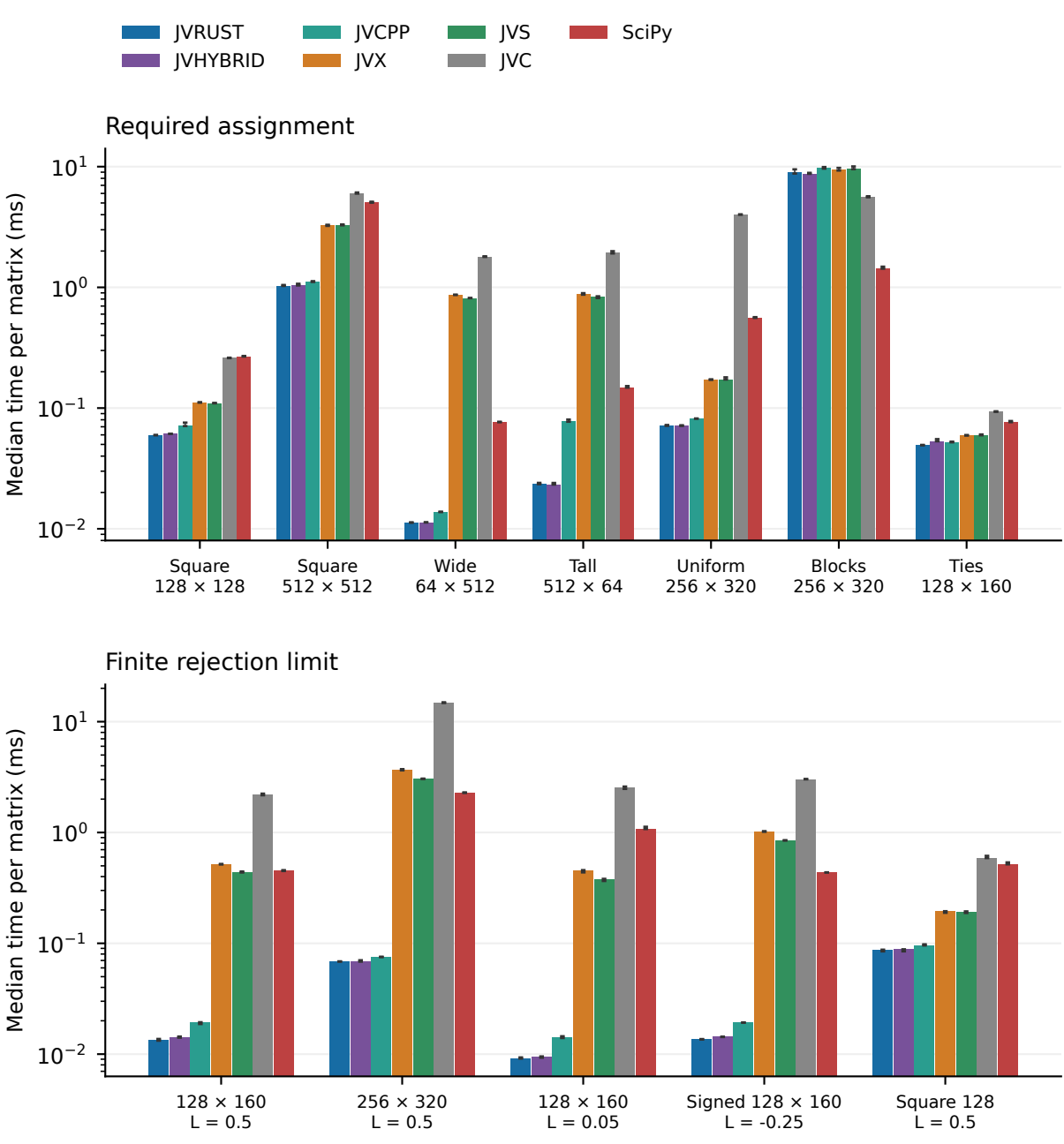
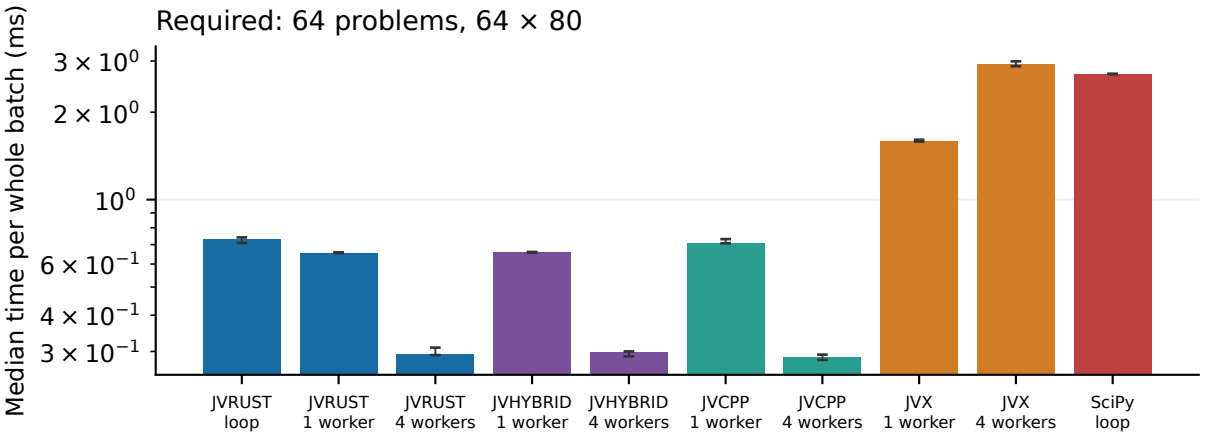
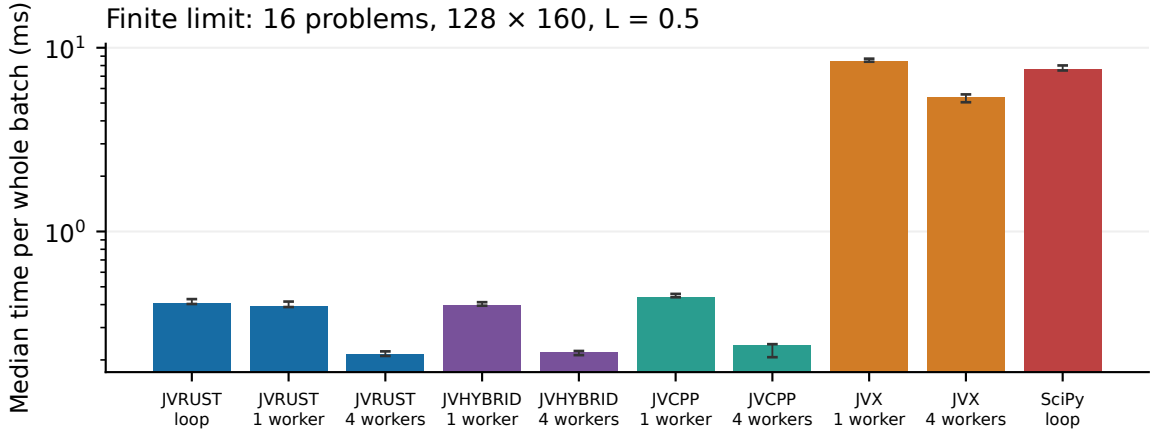
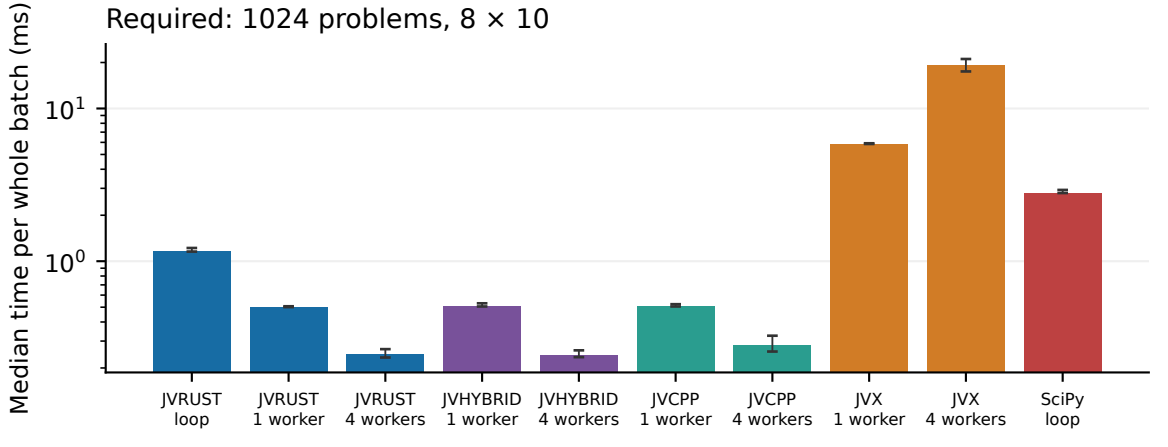
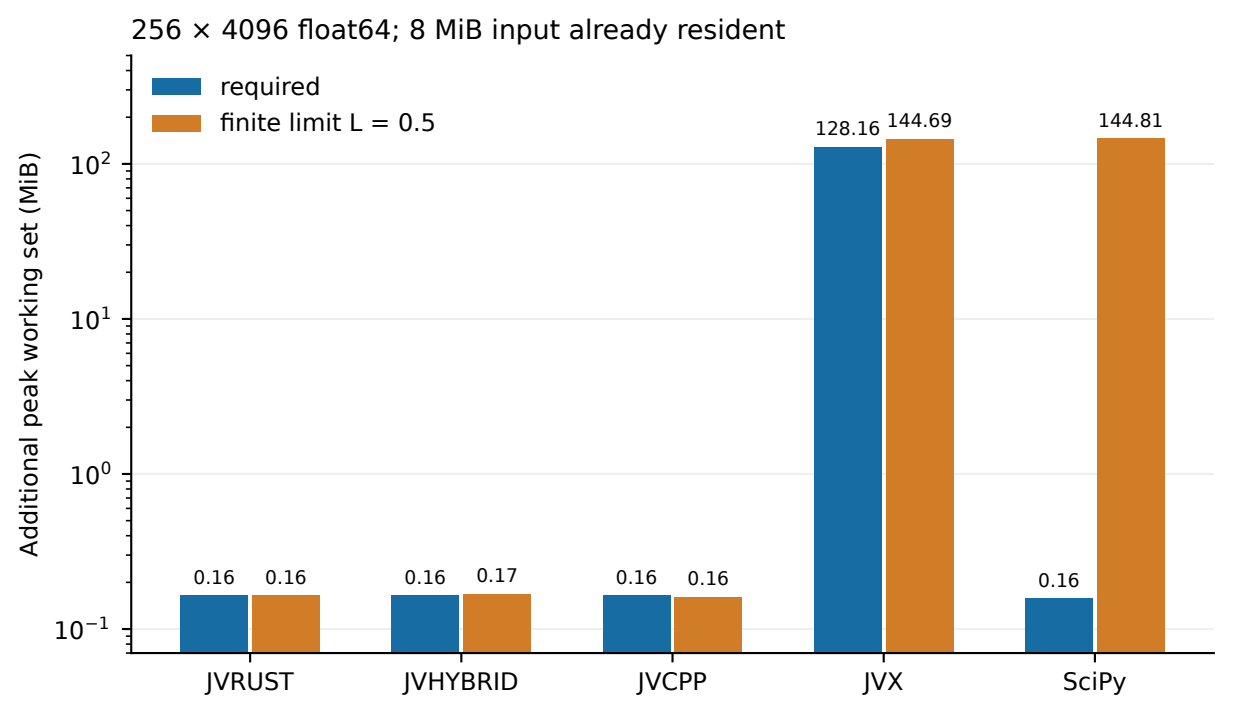
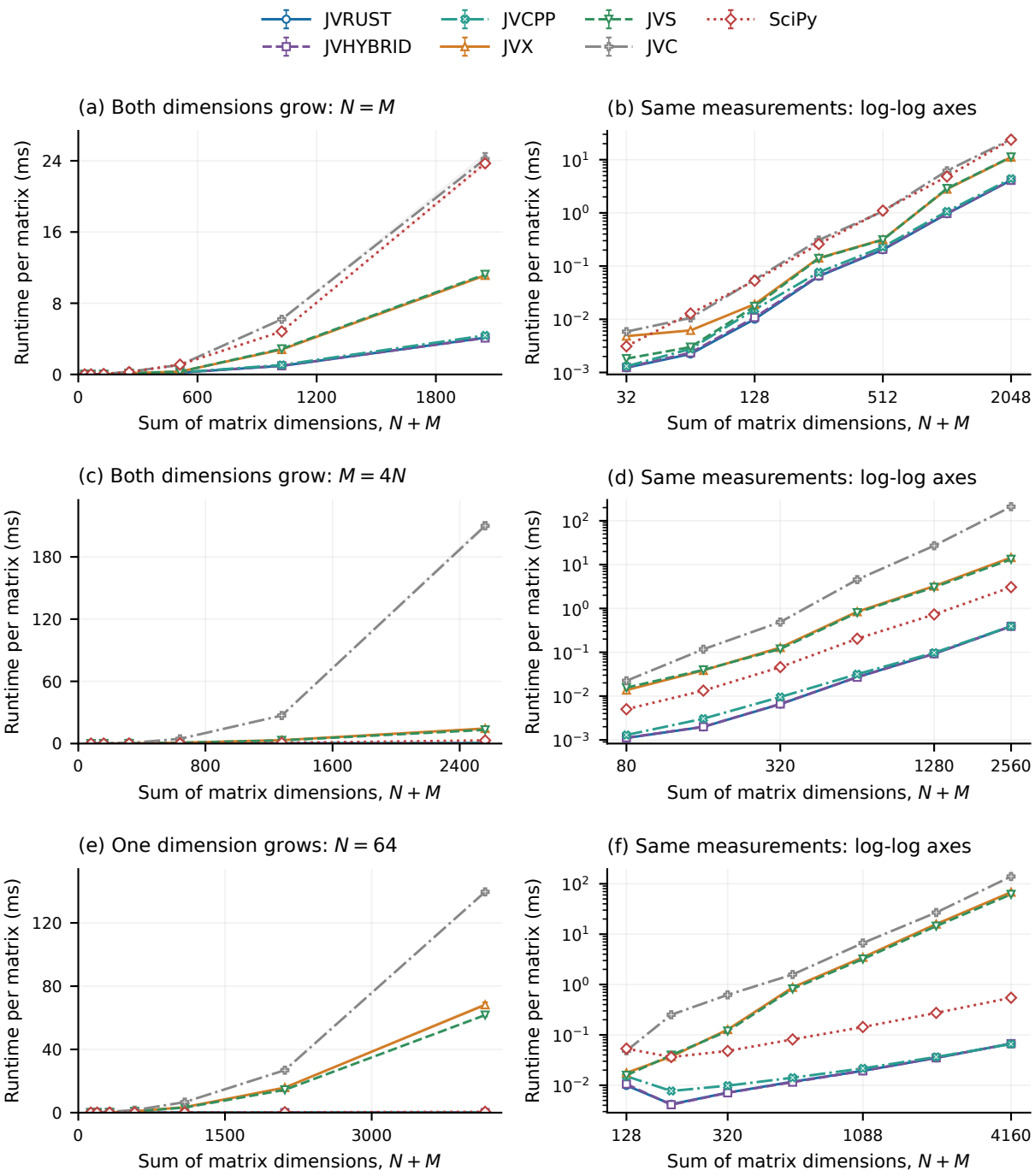

JVRUST
JVHYBRID
JVCPP
JVX
JVS
JVC
SciPy
Required assignment
Median time per matrix (ms)
10^{1}
10^{0}
10^{-1}
10^{-2}
Square
128 × 128
Square
512 × 512
Wide
64 × 512
Tall
512 × 64
Uniform
256 × 320
Blocks
256 × 320
Ties
128 × 160
Finite rejection limit
Median time per matrix (ms)
10^{1}
10^{0}
10^{-1}
10^{-2}
128 × 160
L = 0.5
256 × 320
L = 0.5
128 × 160
L = 0.05
Signed 128 × 160
L = -0.25
Square 128
L = 0.5

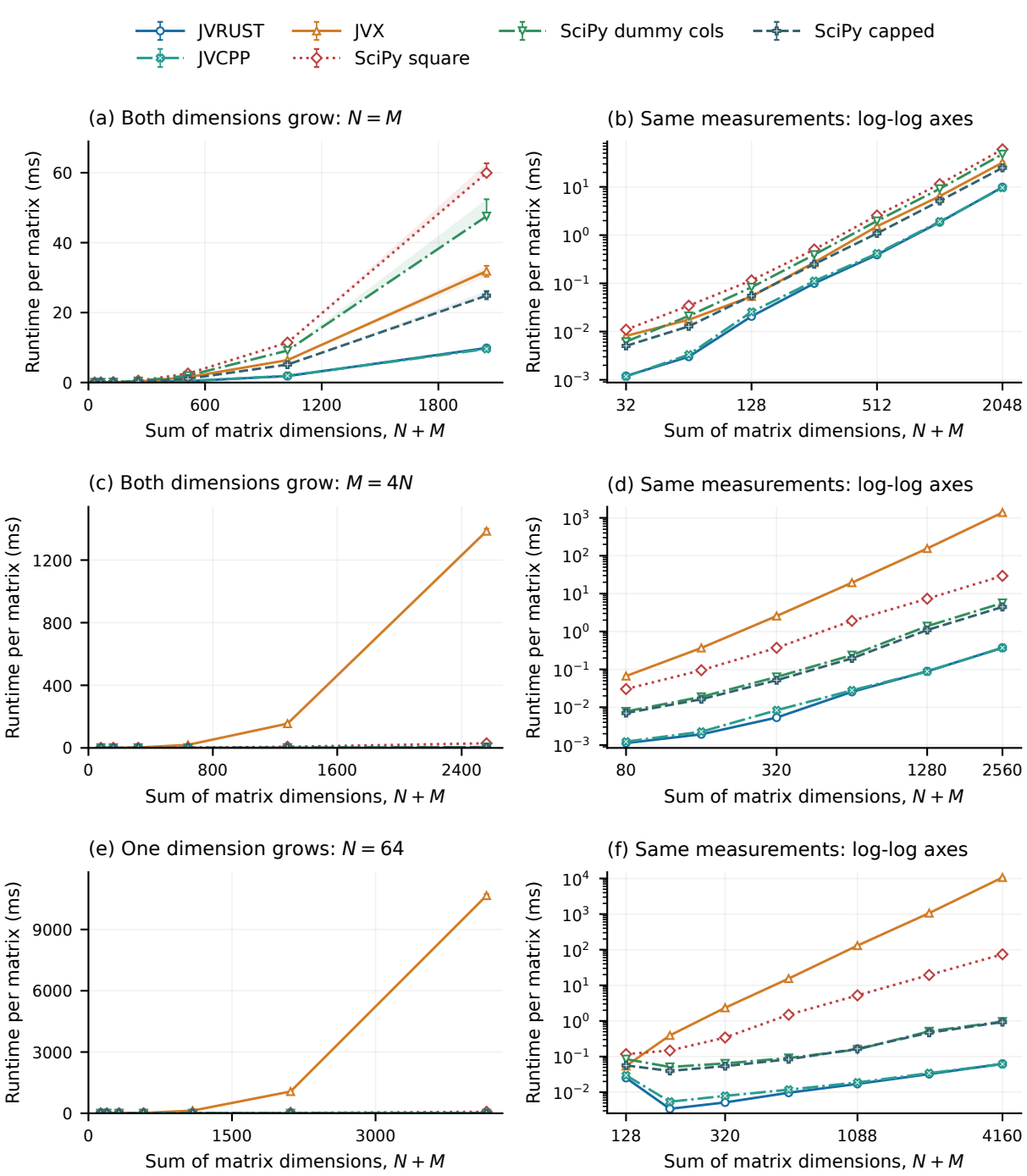

Required: 64 problems, 64 × 80
Median time per whole batch (ms)
3 × 10^0
2 × 10^0
10^0
6 × 10^−1
4 × 10^−1
3 × 10^−1
JVRUST loop
JVRUST 1 worker
JVRUST 4 workers
JVHYBRID 1 worker
JVHYBRID 4 workers
JVCPP 1 worker
JVCPP 4 workers
JVX 1 worker
JVX 4 workers
SciPy loop

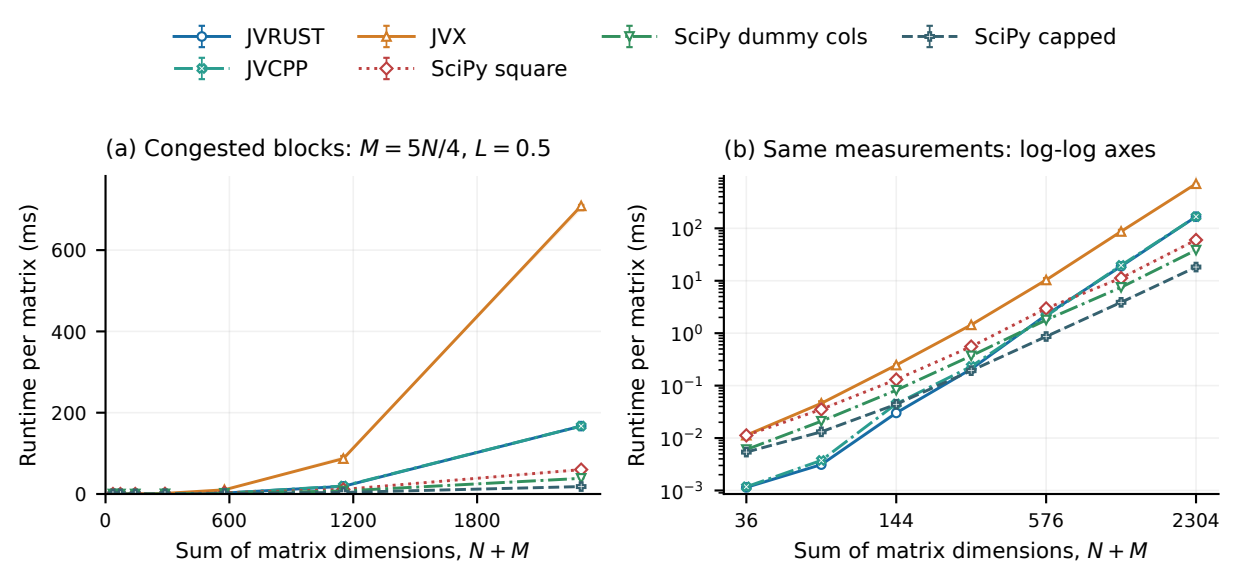

Finite limit: 16 problems, 128 × 160, L = 0.5
Median time per whole batch (ms)
10^1
10^0
JVRUST loop
JVRUST 1 worker
JVRUST 4 workers
JVHYBRID 1 worker
JVHYBRID 4 workers
JVCPP 1 worker
JVCPP 4 workers
JVX 1 worker
JVX 4 workers
SciPy loop

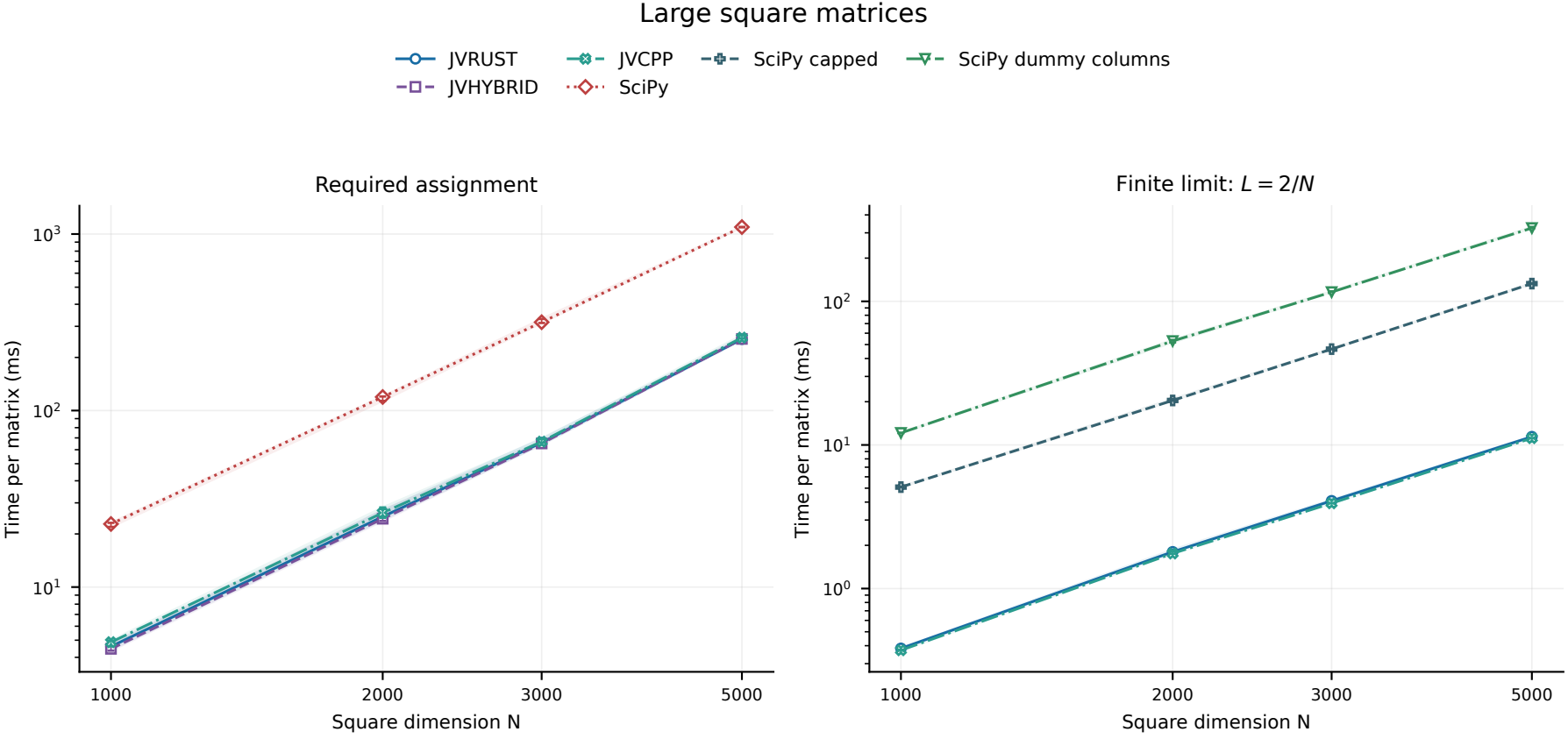

Required: 1024 problems, 8 × 10
Median time per whole batch (ms)
10^1
10^0
JVRUST loop
JVRUST 1 worker
JVRUST 4 workers
JVHYBRID 1 worker
JVHYBRID 4 workers
JVCPP 1 worker
JVCPP 4 workers
JVX 1 worker
JVX 4 workers
SciPy loop


256 × 4096 float64; 8 MiB input already resident
required
finite limit L = 0.5
Additional peak working set (MiB)
10^{2}
10^{1}
10^{0}
10^{-1}
0.16
0.16
0.16
0.17
0.16
0.16
128.16
144.69
0.16
144.81
JVRUST
JVHYBRID
JVCPP
JVX
SciPy

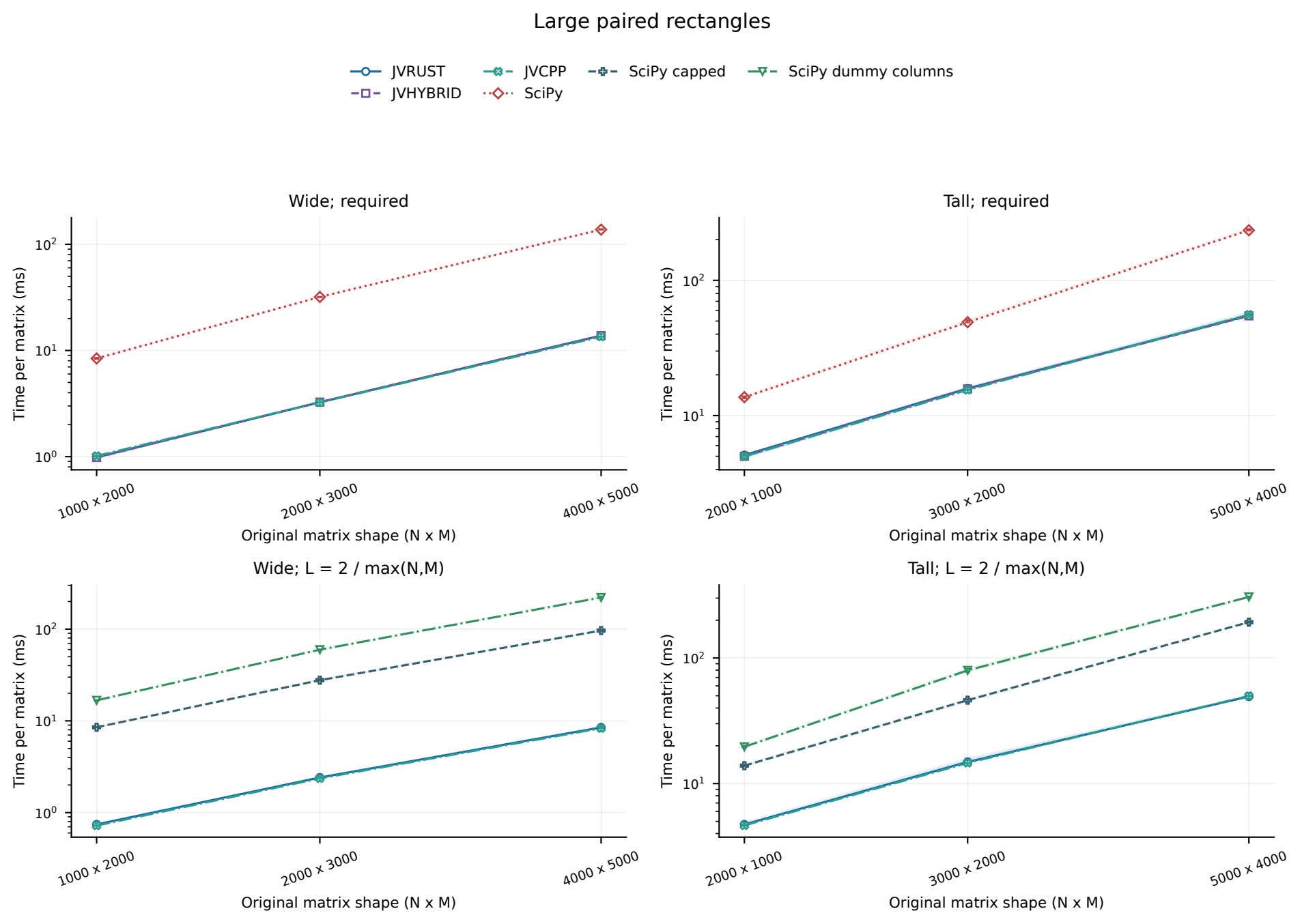

JVRUST
JVHYBRID
JVCPP
JVX
JVS
JVC
SciPy
(a) Both dimensions grow: $N = M$
(b) Same measurements: log-log axes
(c) Both dimensions grow: $M = 4N$
(d) Same measurements: log-log axes
(e) One dimension grows: $N = 64$
(f) Same measurements: log-log axes
Runtime per matrix (ms)
Sum of matrix dimensions, $N + M$


JVRUST
JVCPP
JVX
SciPy square
SciPy dummy cols
SciPy capped
(a) Both dimensions grow: $N = M$
(b) Same measurements: log-log axes
(c) Both dimensions grow: $M = 4N$
(d) Same measurements: log-log axes
(e) One dimension grows: $N = 64$
(f) Same measurements: log-log axes
Runtime per matrix (ms)
Sum of matrix dimensions, $N + M$

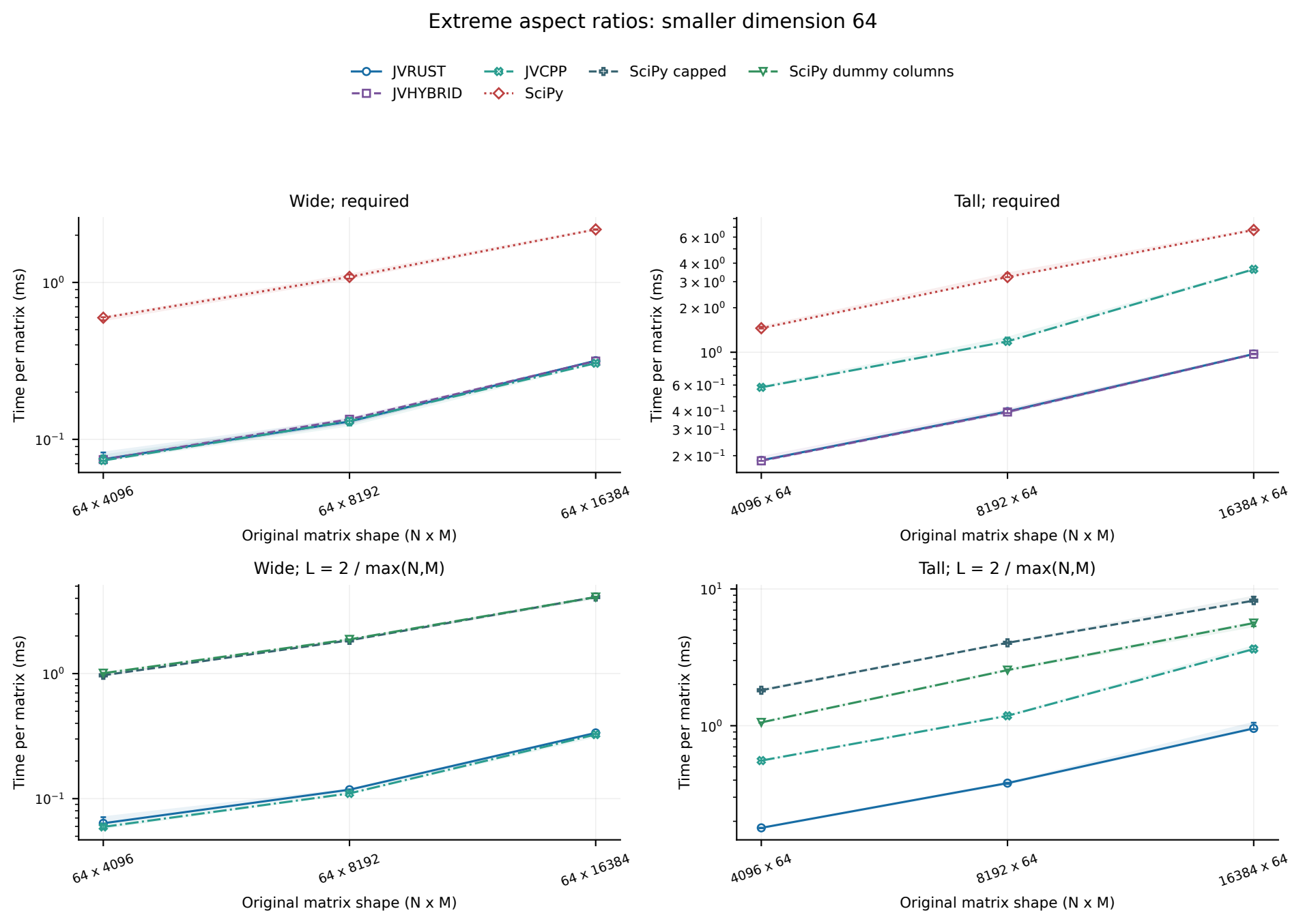

JVRUST
JVCPP
JVX
SciPy square
SciPy dummy cols
SciPy capped
(a) Congested blocks: $M = 5N/4$, $L = 0.5$
Runtime per matrix (ms)
0
200
400
600
0
600
1200
1800
Sum of matrix dimensions, $N + M$
(b) Same measurements: log-log axes
Runtime per matrix (ms)
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
36
144
576
2304
Sum of matrix dimensions, $N + M$



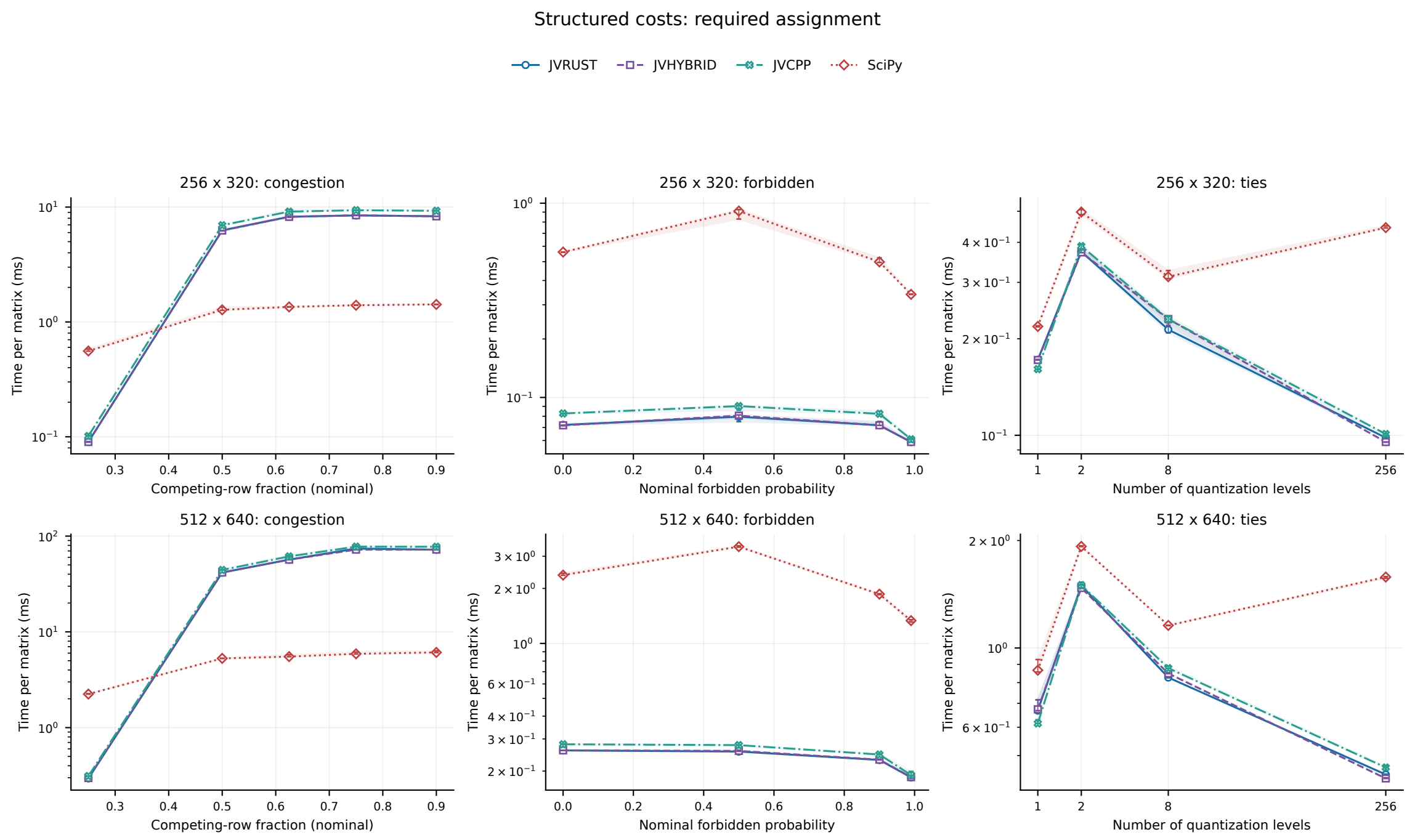
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.



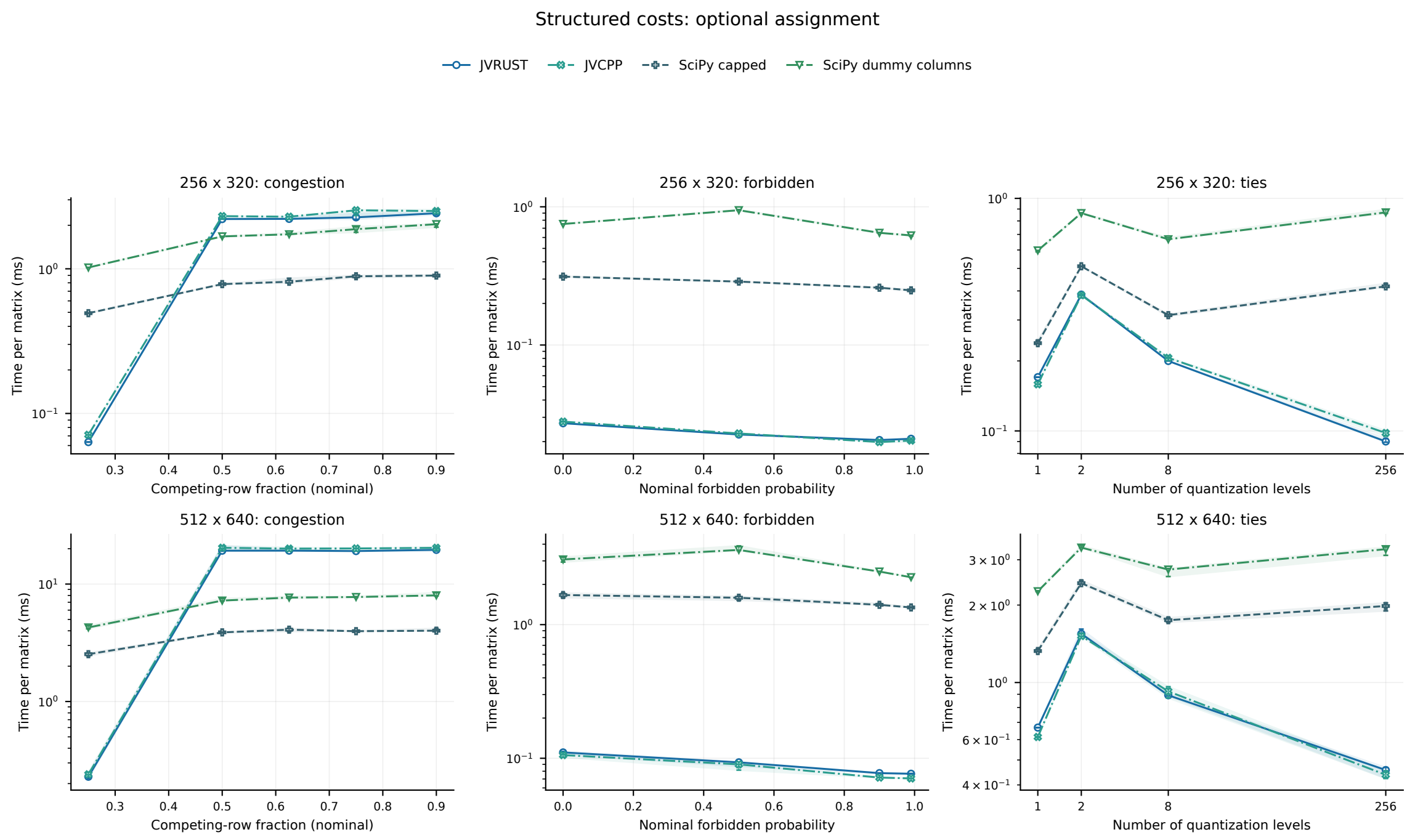
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.
Wide/tall inputs are exact transposes.



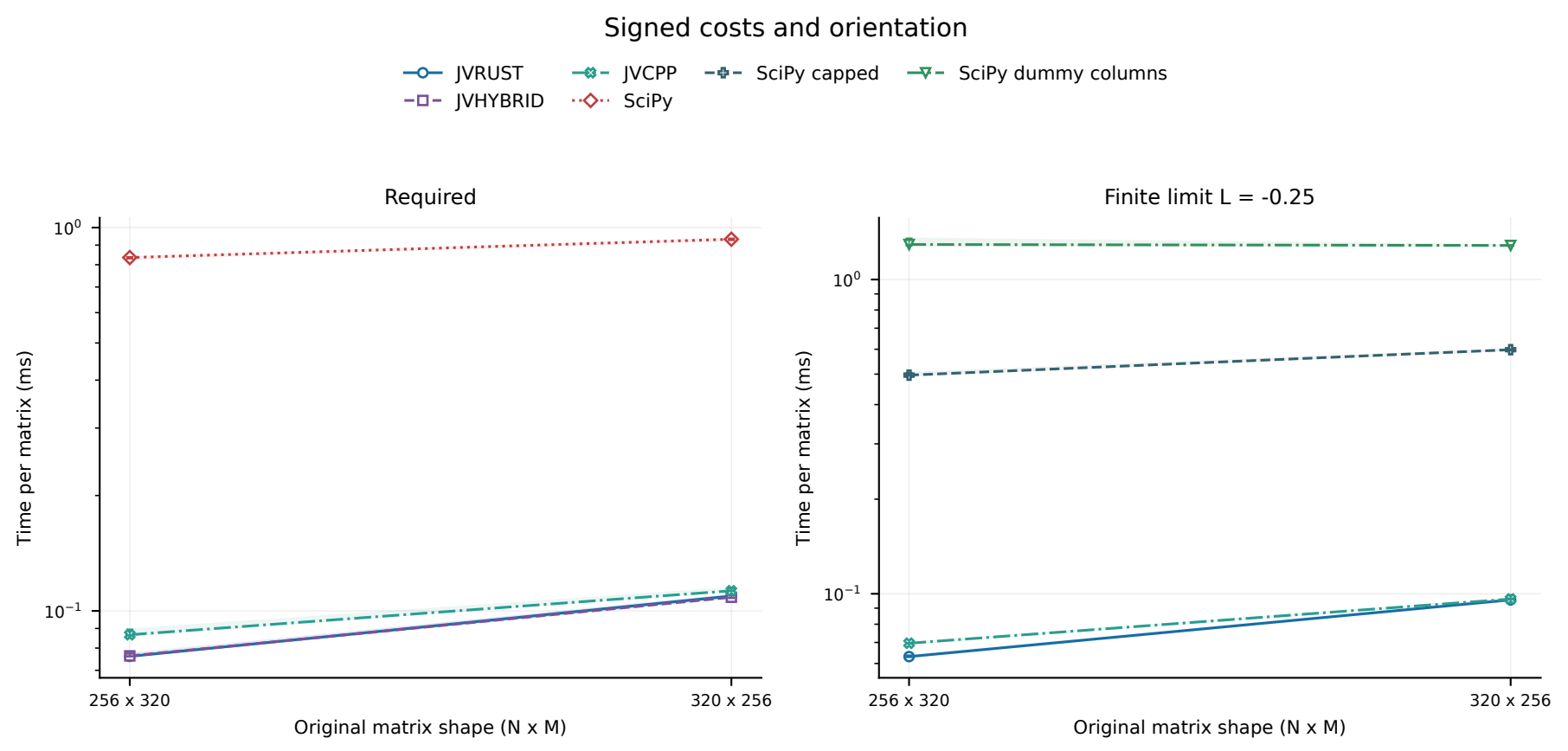
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.
Wide/tall inputs are exact transposes.



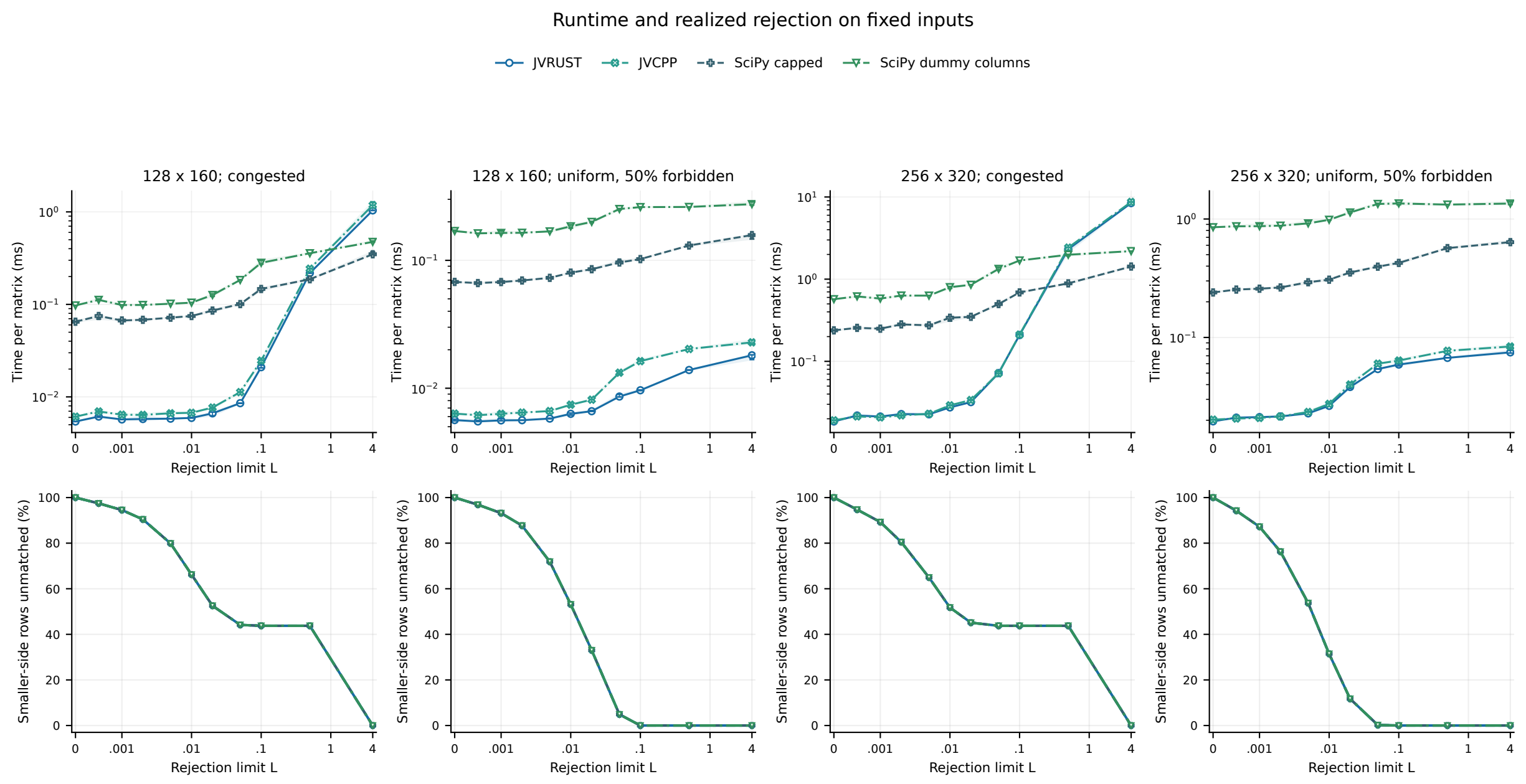
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.
Inputs are paired across each sweep. Forbidden cases retain a feasible diagonal. Optional L: blocks 0.5; forbidden 2/M; ties 0.25.



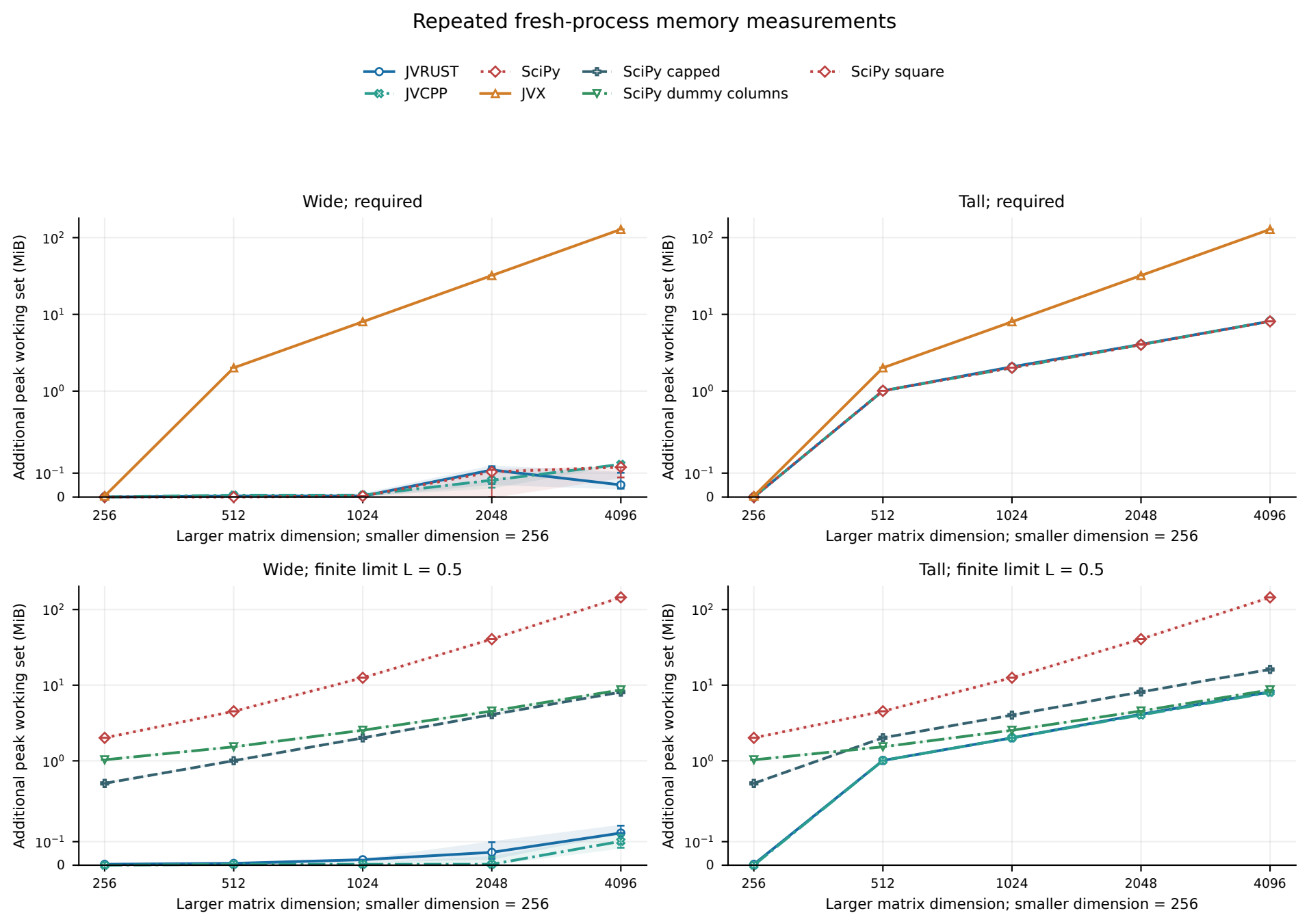
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.
Inputs are paired across each sweep. Forbidden cases retain a feasible diagonal. Optional L: blocks 0.5; forbidden 2/M; ties 0.25.



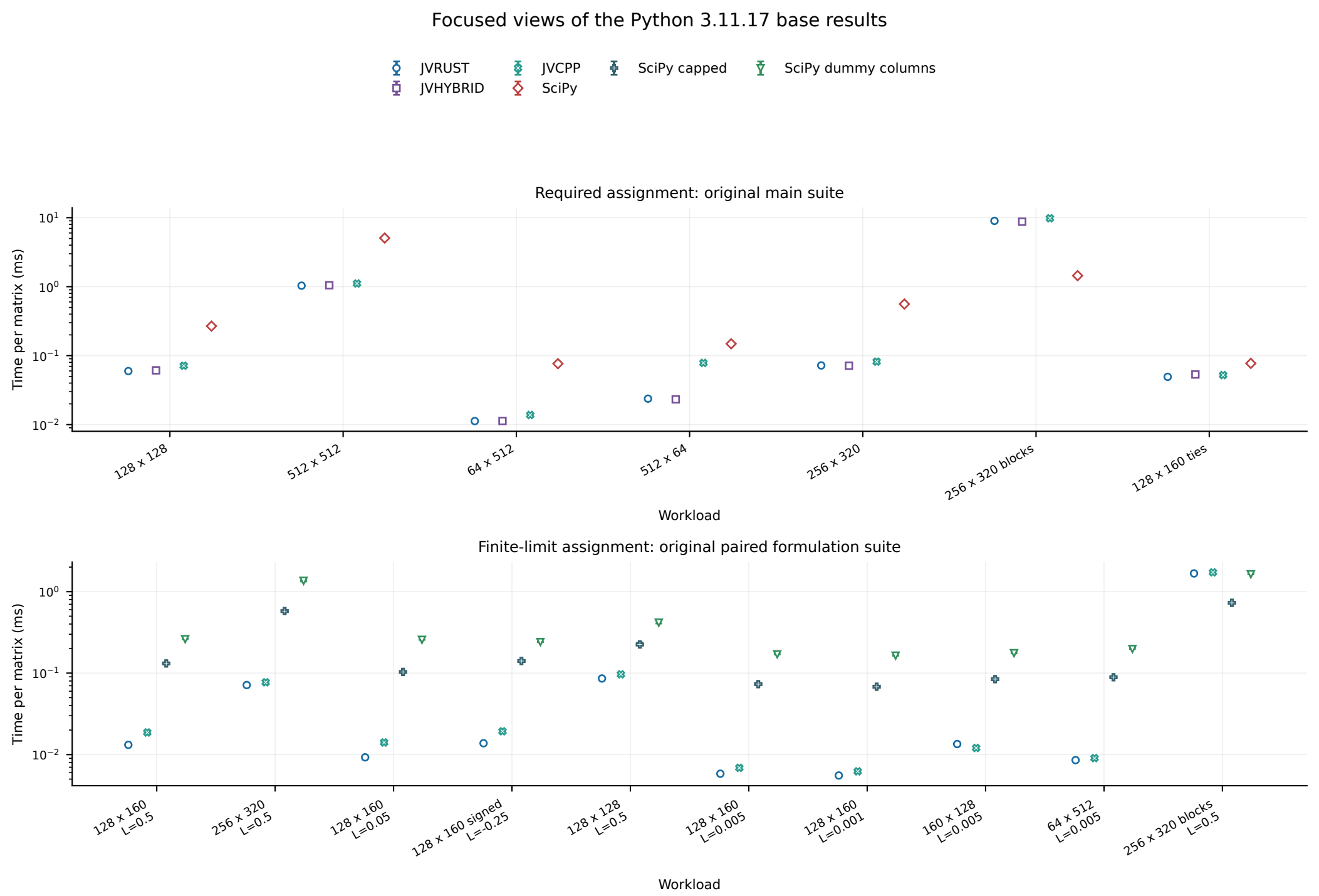
Four paired float64 inputs; nine shuffled rounds. Medians and timing-round IQRs. Focused solver subset; gaps denote exclusions.
Costs in [-1,1), 50% nominal forbidden entries, feasible diagonal retained.



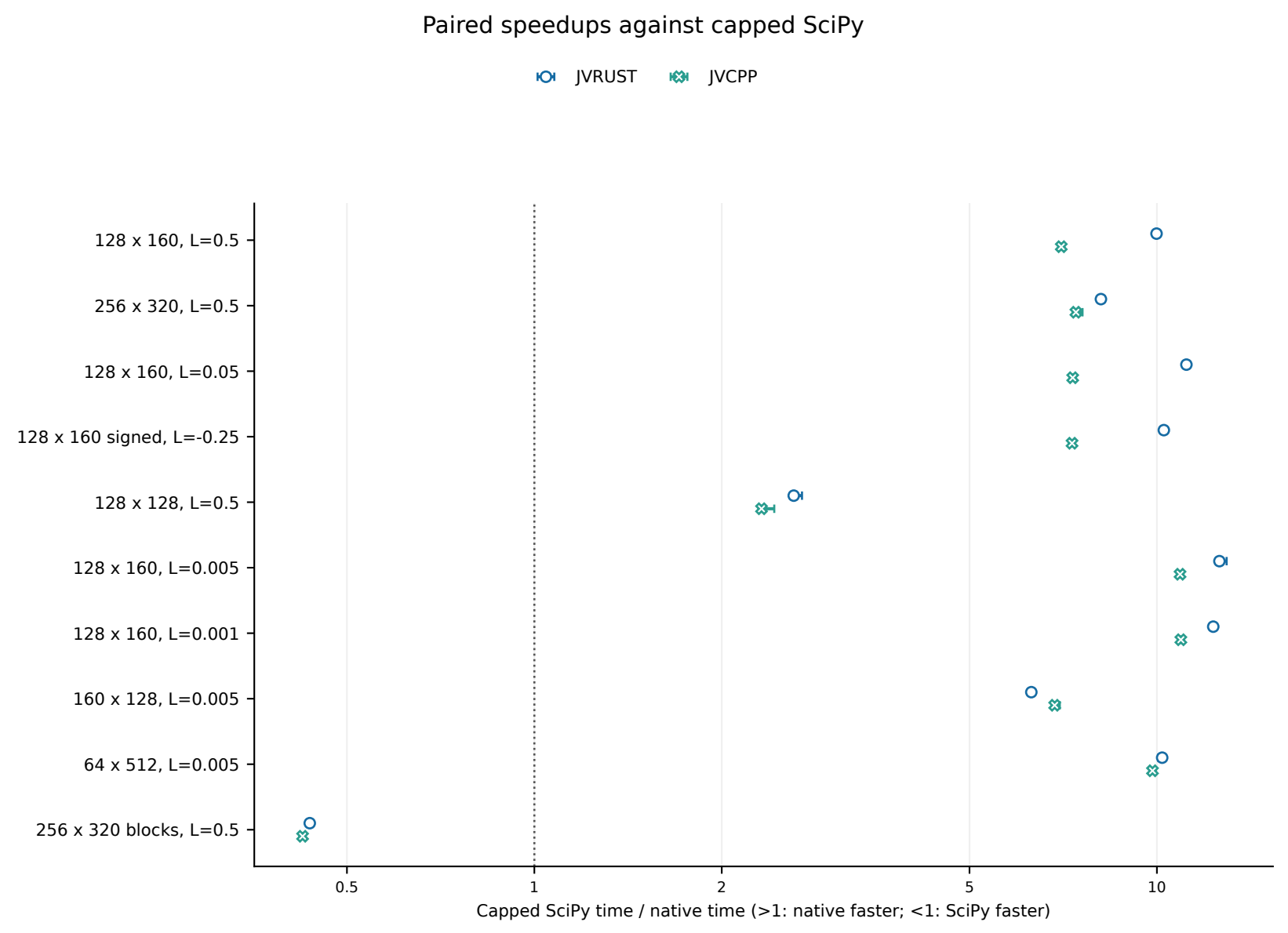
Same four cost matrices and forbidden masks at every limit. Top: median and timing-round IQR. Bottom: mean unmatched fraction.
Different optimal pair sets/cardinalities at ties are permitted; objectives are checked. The limit axis includes zero using a symmetric-log scale.



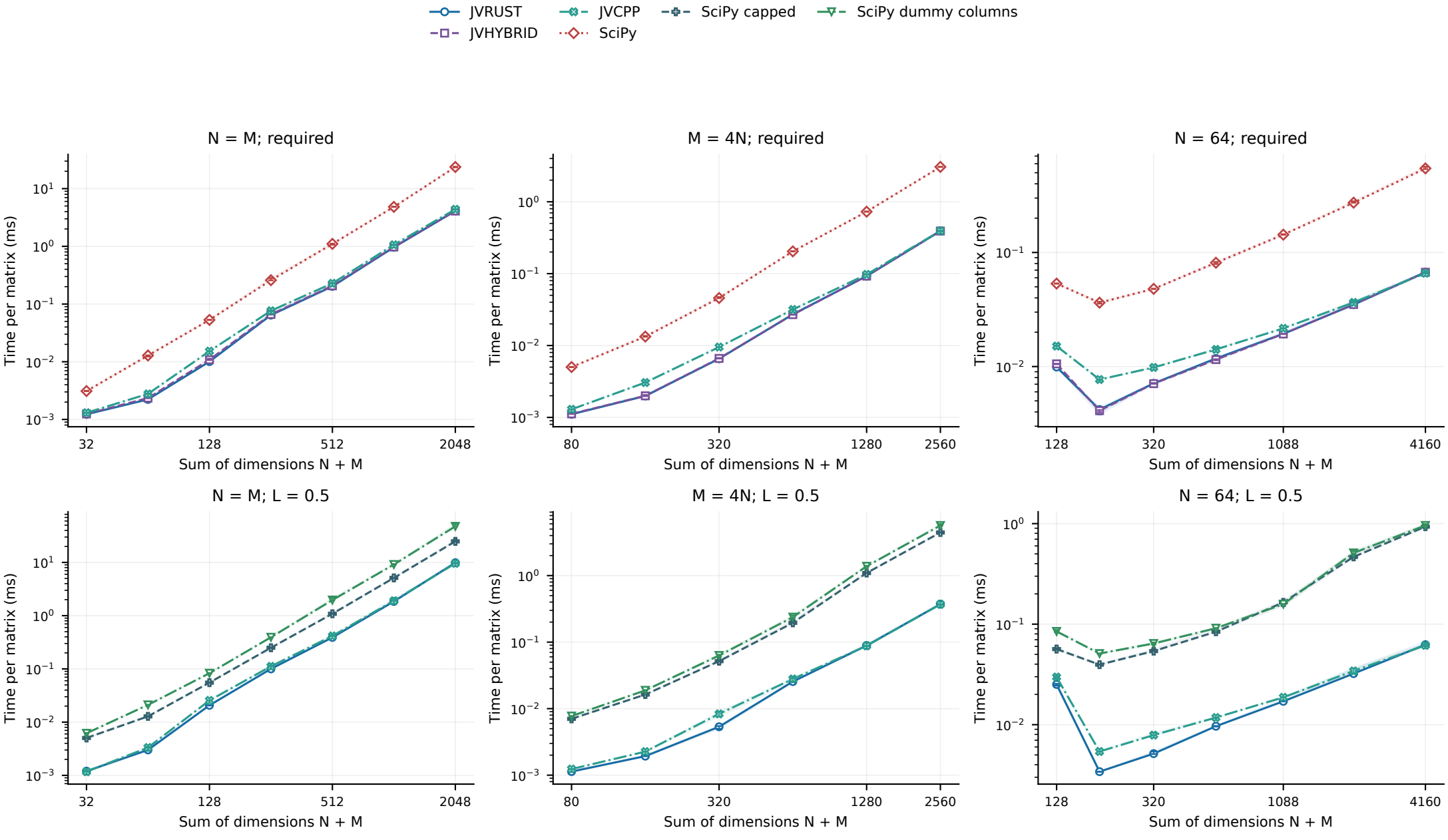
Median and IQR across five fresh processes per condition. Input already resident. High-water increments are not allocation counts.
The vertical scale is linear from zero to 0.25 MiB and logarithmic above it; zero increments remain visible. Compact SciPy formulations are included.



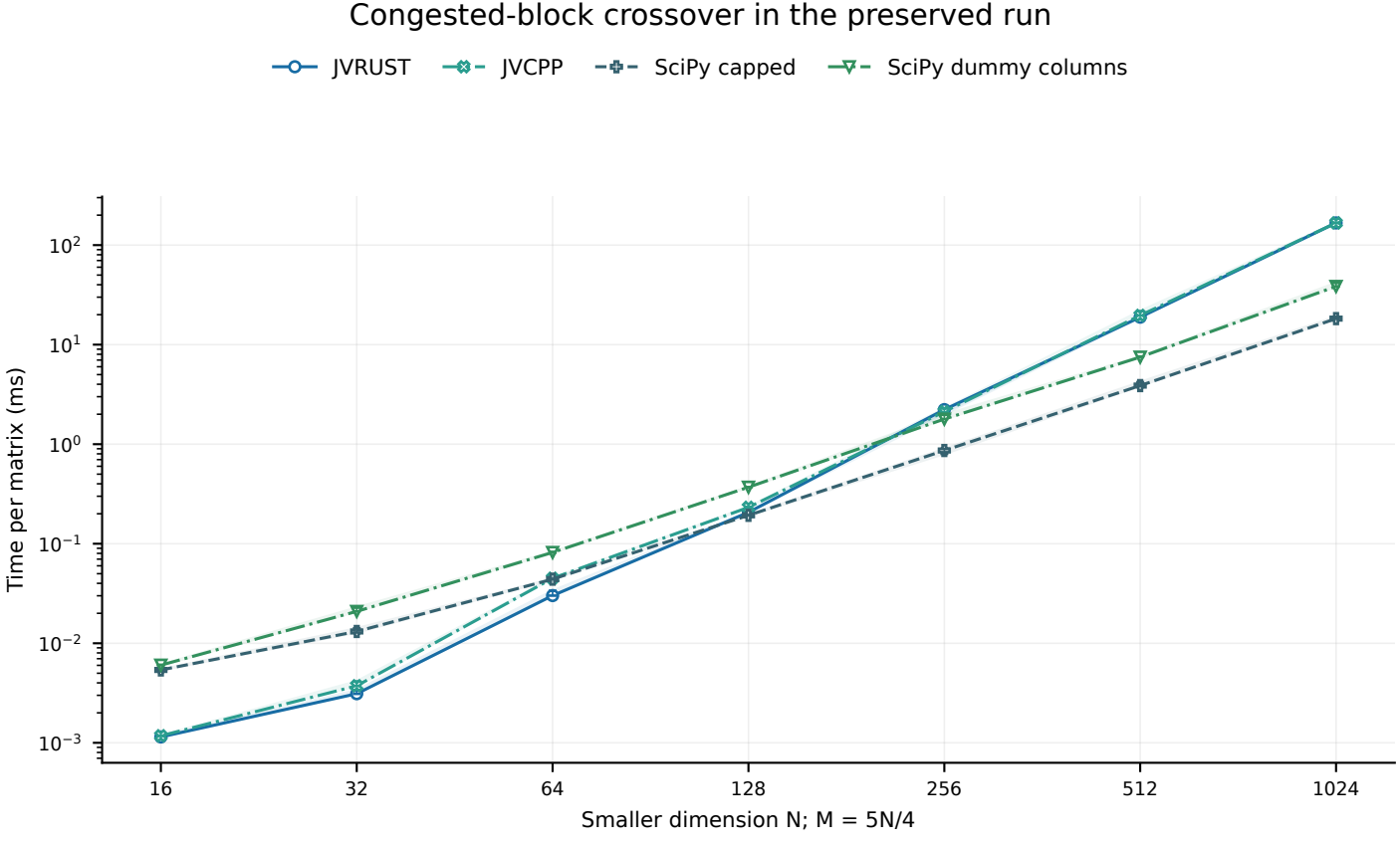
Original measurements retained unchanged; no timings combined across suites. Medians and timing-round IQRs.
The finite-limit panel uses capped and dummy-column SciPy from the paired formulation comparison. Congested cases are retained.



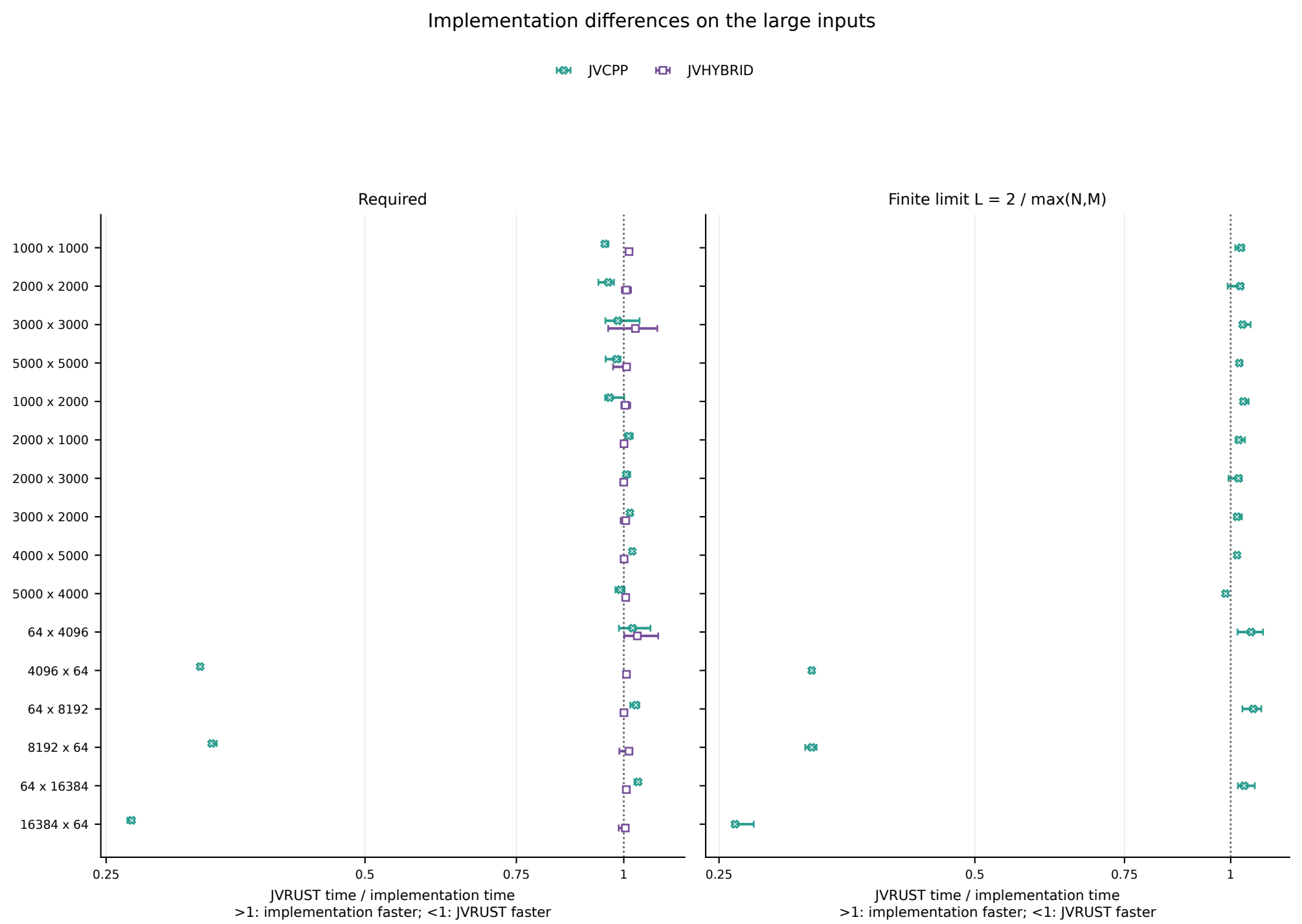
Median and IQR of nine within-round time ratios, using the preserved formulation samples. Ratios are not confidence intervals.
All ten cases are shown, including the congested-block reversal.



Same saved timing samples as the original scaling figures; selected solver subset. Medians and timing-round IQRs.
All original uniform finite-limit instances match the complete smaller side; the new rejection sweep characterizes other regimes.



Original finite-limit sweep, L = 0.5. Selected native and compact SciPy methods; medians and timing-round IQRs.
The full original comparison remains available in the preserved figures and results.



Median and IQR of nine paired round ratios. Same inputs for all implementations; baseline is JVRUST = 1.
Companion large-matrix runtime charts include the SciPy baselines. These ratios compare implementations/builds, not programming languages in general.